

Retrieval Follow-Up Q&A; And Recommendations

****Context****

This document is a follow-up to [retrieval_stack_audit_and_customization_roadmap.pdf](/Users/Work/local_dev/RAG%20workflow/RAG_ENTERPRISE_STARTER/docs/retrieval_stack_audit_and_customization_roadmap.pdf). It captures the next round of retrieval questions that came up after the initial audit and answers them using the current codebase as the source of truth first, then contrasts that behavior with stronger enterprise retrieval patterns where appropriate.

No code is being changed in this document. Where “exact code changes” are shown, they are proposed examples for later implementation.

1. Fusion, Scoring, Reranking, And MMR

1.1 What is the current fusion logic?

Today the core retrieval scoring is implemented in [backend/app/core_rag/retrieval.py](/Users/Work/local_dev/RAG%20workflow/RAG_ENTERPRISE_STARTER/backend/app/core_rag/retrieval.py) and [backend/app/db/repo_search.py](/Users/Work/local_dev/RAG%20workflow/RAG_ENTERPRISE_STARTER/backend/app/db/repo_search.py).

Current score definitions:

• `distance`

- raw PGVector distance from `search\_chunks(...)` - lower is better

• `vector\_score`

- `max(0.0, 1.0 - distance)`

• `rank\_score`

- raw PostgreSQL full-text ranking score from `ts\_rank\_cd(...)`

• `keyword\_score`

- `rank\_score / max\_rank`

• `combined\_score`

- current hybrid formula:

```
combined_score = alpha * vector_score + (1.0 - alpha) * keyword_score
```

Default `alpha` is `0.65` in [backend/app/core/config.py](/Users/Work/local_dev/RAG%20workflow/RAG_ENTERPRISE_STARTER/backend/app/core/config.py), so vector relevance dominates hybrid by default.

Additional score signals already exist:

- `anchor\_score`
- `graph\_score`
- `temporal\_score`
- `rerank\_score`

1.2 What is RRF and why is it often better?

RRF means Reciprocal Rank Fusion. It does not add raw vector and keyword scores together. Instead, it fuses rank positions:

$$\text{RRF}(\text{document}) = \sum \frac{1}{(k + \text{rank}_i(\text{document}))}$$

Typical `k = 60`.

Why it is usually more stable than linear fusion:

- vector and lexical scores do not need to be on comparable scales
- exact lexical wins are less likely to get buried
- it is less sensitive to score normalization mistakes
- adding another retriever later does not force score-rescaling logic

1.3 Example

If vector ranks are:

- A rank 1
- B rank 2
- C rank 3

And keyword ranks are:

• C rank 1

• A rank 2

• D rank 3

Then with $k = 60$:

• $A = \frac{1}{61} + \frac{1}{62}$

• $C = \frac{1}{63} + \frac{1}{61}$

• $B = \frac{1}{62}$

• $D = \frac{1}{63}$

Documents supported by multiple retrievers rise naturally.

1.4 Proposed code change for RRF

This is not applied, but it is the direct shape of the future change:

```
diff --git a/backend/app/core/config.py b/backend/app/core/config.py
@@
+ HYBRID_ALPHA: float = 0.65
+ HYBRID_FUSION_METHOD: str = "linear"
+ RRF_K: int = 60

diff --git a/backend/app/core_rag/retrieval.py b/backend/app/core_rag/retrieval.py
@@
-def merge_hybrid_results(vector_results, keyword_results, k, alpha=0.65):
+def merge_hybrid_results(vector_results, keyword_results, k, alpha=0.65, fusion_method="linear", rrf_k=60):
+ if fusion_method == "rrf":
+ return merge_hybrid_results_rrf(vector_results, keyword_results, k, rrf_k)
@@
+def merge_hybrid_results_rrf(vector_results, keyword_results, k, rrf_k=60):
+ merged = {}
+ for rank, result in enumerate(vector_results, start=1):
+ chunk_id = result["chunk_id"]
+ merged.setdefault(chunk_id, {"vector_score": 0.0, "keyword_score": 0.0})
+ merged[chunk_id]["vector_score"] = 1.0 / (rrf_k + rank)
+ for rank, result in enumerate(keyword_results, start=1):
+ chunk_id = result["chunk_id"]
+ merged.setdefault(chunk_id, {"vector_score": 0.0, "keyword_score": 0.0})
```

```

+         merged[chunk_id]["keyword_score"] = 1.0 / (rrf_k + rank)
+     final_results = []
+     for result in merged.values():
+         result["combined_score"] = result["vector_score"] + result["keyword_score"]
+         final_results.append(result)
+     final_results.sort(key=_result_sort_key)
+     return final_results[:k]

```

1.5 What is MMR?

MMR means Maximal Marginal Relevance. It is not the same thing as reranking.

- reranking: reorder documents by query relevance

- MMR: reorder documents to balance relevance and diversity

Conceptually:

$$\text{MMR} = \lambda * \text{relevance_to_query} - (1 - \lambda) * \text{similarity_to_selected_results}$$

The repo does not implement MMR today.

1.6 Why is reranking optional?

Reranking exists in [backend/app/core_rag/reranker.py](/Users/Work/local_dev/RAG%20workflow/RAG_ENTERPRISE_STARTER/backend/app/core_rag/reranker.py). It uses:

- `cross-encoder/ms-marco-MiniLM-L-6-v2`

Why it is optional:

- it adds latency

- it adds model runtime cost

- some corpora do not need it on every query

- development environments often prioritize speed

Current repo behavior:

- reranking is an admin/config switch

- it is not exposed to end users as a feedback-driven runtime option
- there is no current logic like “turn reranking on for this user because they previously marked results bad”

1.7 Implication for this repo

- keep linear fusion as baseline-safe until RRF lands
- add RRF as a configurable alternative
- keep reranking optional at system level, not a raw end-user toggle
- add MMR later only if repeated near-duplicate top results become a real issue

2. Chunking, Lexical Retrieval, Anchors, Heuristics, And Query Transformation

2.1 What does “heuristic” mean?

A heuristic is a practical rule-of-thumb, not a guaranteed-correct learned decision.

Repo-specific examples:

- router keyword intent detection in `[backend/app/core_rag/query_router.py](/Users/Work/local_dev/RAG%20workflow/RAG_ENTERPRISE_STARTER/backend/app/core_rag/query_router.py)`
- anchor extraction in `[backend/app/core_rag/retrieval.py](/Users/Work/local_dev/RAG%20workflow/RAG_ENTERPRISE_STARTER/backend/app/core_rag/retrieval.py)`
- fixed chunk sizes and overlaps in `[backend/app/ingestion/chunking.py](/Users/Work/local_dev/RAG%20workflow/RAG_ENTERPRISE_STARTER/backend/app/ingestion/chunking.py)`

Why it is repeatedly called a limitation:

- heuristics are easy to explain and fast to run
- but they can fail on edge cases
- they are usually not confidence-aware or domain-adaptive

2.2 What is lexical retrieval?

Lexical retrieval means matching based on words/tokens rather than semantic meaning.

In this repo, lexical retrieval is the `keyword` path using PostgreSQL full-text search in [backend/app/db/repo_search.py](/Users/Work/local_dev/RAG%20workflow/RAG_ENTERPRISE_STARTER/backend/app/db/repo_search.py).

It helps most for:

- exact phrases
- names
- document IDs
- codes and SKUs
- quoted text
- dates and version strings

This capability should exist by default in enterprise RAG. The question is usually not whether to have lexical retrieval, but whether it should be:

- standalone
- blended inside hybrid retrieval
- or made dominant for some query classes

2.3 What is adaptive chunking?

Adaptive chunking means chunk shape changes based on:

- document type
- corpus policy
- content structure
- or query class

The repo already has document-type-specific chunking, which is a primitive form of adaptive chunking:

- PDF page and cross-page chunks

- DOCX heading-aware buffering
- PPTX slide chunks
- XLSX row-range chunks
- EML header/body chunks
- TXT and Markdown text/section chunks

But it does not yet do:

- tokenizer-aware chunking
- query-adaptive chunk sizing
- learned chunk boundary detection

It can absolutely be used selectively. That is the preferred rollout path.

2.4 What are query anchors?

Query anchors are token-rule-based lexical cues extracted from the question or admin-supplied input. They are defined through hardcoded logic in `[backend/app/core_rag/retrieval.py](/Users/Work/local_dev/RAG%20workflow/RAG_ENTERPRISE_STARTER/backend/app/core_rag/retrieval.py)`, not through LLM reasoning.

So yes:

- they are useful
- they are cheap
- they can fail

2.5 What is richer lexical pattern detection?

This usually belongs in the router stage before retrieval.

Examples:

- quote / exact wording requests
- final words / last sentence / exact phrase

- SKU, case number, contract number, invoice number
- date-heavy exact lookup

It can be implemented with:

- hard-coded patterns and regexes
- lightweight classifier
- optional model assistance

Today this repo only has heuristic hard-coded logic.

2.6 What about query rewriting, expansion, and HyDE?

These are absent as full features.

Current partial substitutes:

- `custom\_query`
- `exact\_phrase\_bias`
- `anchor\_terms`
- deep-research anchor query generation

These are not equivalent to:

- LLM query rewrite
- expansion to alternate phrasings
- hypothetical-document generation (HyDE)

2.7 Implication for this repo

- keep lexical retrieval as a first-class default capability
- strengthen lexical intent detection before adding expensive rewrite layers
- treat heuristics as good baseline infrastructure, not final enterprise behavior

3. Email, EML, Attachments, And Enterprise Ingestion Realities

3.1 Is EML the accepted format?

Yes. `eml` is explicitly supported in [backend/app/core/config.py](/Users/Work/local_dev/RAG%20workflow/RAG_ENTERPRISE_STARTER/backend/app/core/config.py) and parsed in [backend/app/adapters/email/parser.py](/Users/Work/local_dev/RAG%20workflow/RAG_ENTERPRISE_STARTER/backend/app/adapters/email/parser.py).

3.2 Are attachments searchable today?

Not as searchable child-source content by default.

What exists today:

- attachment metadata is parsed
- attachment records exist in schema
- attachment links can be stored

What does not exist today:

- automatic attachment content parsing into child searchable sources
- automatic attachment indexing inside the main retrieval flow

This is also documented in the parser capability docs.

3.3 Is EML enough for enterprise email?

Usually no.

In enterprises, email often lives as:

- Exchange / Microsoft 365 mailboxes
- Gmail / Google Workspace

- PST / OST archives
- journaling systems
- eDiscovery exports
- archive connectors

So your assumption is correct: in real enterprise workflows, almost nobody uploads `.eml` manually as the core ingestion path.

3.4 What should the enterprise path look like?

Future-state email ingestion should look like:

1. mailbox/archive connector
2. normalized email document model
3. thread metadata and participant metadata
4. attachment-as-child-source ingestion for supported file types
5. parent-child linkage for retrieval and provenance

3.5 Implication for this repo

- current `.eml` support is a valid PoC feature
- enterprise readiness requires connector-backed email ingestion and attachment child-source indexing

4. Metadata, Filtering, Temporal Grounding, And Clarification Flows

4.1 How is metadata used today?

Current metadata usage is split into:

- explicit filters
 - `source\_type` - `source\_id` - `source\_part\_id` - `locator\_filter`
- temporal metadata
 - extracted and used as query-time boost/rerank in `full`
- graph metadata
 - used for graph-aware candidate support

- source metadata JSON

- stores graph, temporal, and lazy-enrichment state

4.2 Does the repo convert English filters automatically?

Not broadly.

The repo has:

- explicit metadata filters

- temporal signal detection

- graph signal detection

- query routing

But it does not yet do broad English-to-filter transformation like:

- “last two years” -> computed date window filter

- “latest contract” -> metadata constraint

- “in Europe only” -> normalized region filter

4.3 Example: “last two years”

A future enterprise retrieval path could do:

1. ground today's date
2. compute the date window
3. rewrite the query or attach structured filters
4. keep the concrete date window visible in the final answer

That is a recommendation, not current functionality.

4.4 What about clarification popups?

There is no clarification UX in the repo today.

Future clarification prompts could help with:

- spelling mistakes
- ambiguous entities
- inconsistent dates
- no exact lexical match for a probable code/ID
- deep-research escalation

Typical behavior would be:

- before retrieval for obvious ambiguity
- after low-confidence first retrieval
- after date grounding if the interpretation is risky

4.5 Implication for this repo

- explicit filtering exists
- inferred metadata interpretation is partial
- clarification is a product feature for later, not current backend behavior

5. Semantic Caching, Budget-Aware Logic, Latency, And Observability

5.1 What is semantic caching?

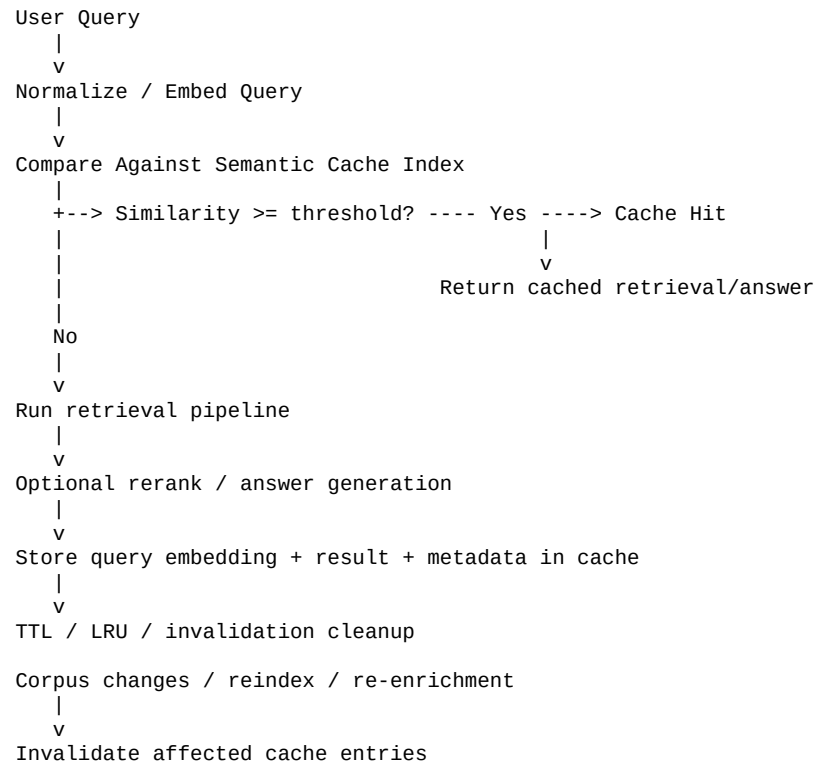
Semantic caching means storing prior query results and reusing them when a new query is semantically close enough.

This is different from what exists today:

- embedding model singleton caching
- reranker model singleton caching

Those are process-level object caches, not semantic query-result caches.

5.2 Diagram



5.3 Would admins control cleanup?

In a future system, yes, likely through:

- TTL
- LRU
- maximum cache size
- manual purge
- purge on reindex
- per-corpus or per-tenant cache scope

5.4 What is budget-aware logic?

It includes:

- latency budget
- token budget
- rerank compute budget
- retrieval-depth budget
- graph/temporal enrichment cost
- infrastructure budget

So yes, it is much broader than “token cost only.”

5.5 Can latency be shown in the UI?

Yes. The repo already returns `latency\_ms` and the frontend already displays latency in [frontend/ask.js](/Users/Work/local_dev/RAG%20workflow/RAG_ENTERPRISE_STARTER/frontend/ask.js).

5.6 Can latency logs be compared against accuracy and feedback later?

Yes, and that is one of the best future tuning loops:

- log latency per stage
- log strategy/mode
- log answer outcome
- log user feedback
- compare latency vs perceived quality vs groundedness

5.7 Hypothetical latency budget table

Stage	Typical Role	Example Time
Query parsing / routing	detect mode and signals	5-20 ms

Stage	Typical Role	Example Time
Query embedding	dense embedding generation	15-80 ms
Vector search	candidate fetch	10-80 ms
Keyword search	FTS candidate fetch	5-40 ms
Hybrid merge / boosts	fusion, anchors, graph/temporal boosts	1-15 ms
Deep-research extras	anchor scans, neighbor expansion	20-120 ms
Cross-encoder rerank	rerank top candidates	40-250 ms
Answer LLM	grounded answer generation	400-3000+ ms
Answer repair	second pass if needed	300-1500+ ms
Fast path total	search + answer	500-1200 ms
Rich path total	rerank + repair + deep lookup	1000-4000+ ms

5.8 What counts as good latency?

There is no single global standard, but practical enterprise expectations are roughly:

- under 1 second: excellent
- 1-2 seconds: very good
- 2-5 seconds: acceptable for rich grounded answers if clearly justified
- above 5 seconds: needs progress UI, deep-research framing, or policy tuning

5.9 Implication for this repo

- latency should be treated as stage-level, not one number
- semantic caching is absent but well-motivated
- budget-aware routing is a future orchestration layer, not current behavior

6. Evaluation, User-Query Mining, Admin Controls, And Future UI Design

6.1 What does “LVE user-query mining” likely mean?

Practically, it means:

- collect real user queries
- capture retries and failures
- cluster query patterns
- annotate meaningful samples
- track drift over time
- track acceptance and operator-visible quality

This full loop does not exist today.

6.2 What exists today?

The repo already has:

- retrieval eval harness
- answer eval harness
- compare eval harness
- benchmark fixtures

But it does not have:

- live query mining
- annotation workflow
- drift tracking
- acceptance metrics
- online experimentation loop

6.3 What would a rich admin UI eventually expose?

Examples:

- retrieval mode defaults
- fusion method and parameters
- rerank policy and candidate depth
- deep-research defaults
- semantic cache settings
- query transformation toggles
- chunking policy by corpus
- graph/temporal toggles
- latency dashboard
- retrieval path comparison
- eval benchmark comparisons
- failure-cluster dashboard

6.4 Caution

Too many knobs can make a system look smart while becoming unstable.

So the admin UX should separate:

- safe defaults
- expert knobs
- experimental knobs

6.5 Implication for this repo

- current offline eval base is useful
- future enterprise maturity depends on real query mining, annotation, and latency/quality observability

What exists today vs what is missing

Present

- document-type-specific chunking
- dense embeddings
- keyword/vector/hybrid retrieval
- heuristic router
- graph/temporal retrieval layers
- optional cross-encoder reranking
- latency in backend response and frontend display
- fixture-based offline evaluation

Partial

- metadata usage
- exact-lookup detection
- deep-research retrieval rescue
- answer repair
- admin visibility into retrieval defaults
- attachment handling metadata only

Absent

- RRF
- MMR
- semantic caching
- full query rewrite / expansion / HyDE
- attachment content ingestion from emails
- connector-backed enterprise email ingestion

- clarification UX
- broad English-to-filter transformation
- real user query mining
- annotation flow
- drift tracking
- acceptance metrics
- feedback-driven routing/reranking policy

Recommended next moves in order

1. Add RRF as configurable fusion alongside current linear fusion. 2. Improve router lexical-intent detection for quote/factoid/code/date-heavy queries. 3. Add rerank policy controls, then consider enabling reranking by default outside dev. 4. Add stage-level latency logging and feedback correlation. 5. Add enterprise email ingestion design: connectors plus attachment-as-child-source parsing. 6. Add semantic cache with TTL and invalidation rules. 7. Add query rewrite/HyDE only behind explicit flags and with offline evaluation. 8. Build real user-query mining, labeling, and drift tracking before adding too many “smart” retrieval behaviors.